

Lab 3: Component Modelling & Architectural Pattern Selection

Telemedicine Slot Booking & Prescription Portal

Field	Details
Student Name	Sriranga Bharadwaj
SRN	PES1UG24AM286
Domain	Healthcare & Telemedicine
Assigned Actors	Patient, Doctor, Clinic Administrator
Lab	Lab 3 — Component Modelling & Architectural Pattern Selection

1. Scenario Understanding & Requirement Analysis

1.1 Problem Statement

The Telemedicine Slot Booking & Prescription Portal is a healthcare platform that allows patients to discover doctors, book available consultation slots, and receive digital prescriptions issued by doctors after a consultation. The system must support two primary user roles — Patients and Doctors — with an underlying administrative layer for clinic/registrar-level oversight (managing doctor onboarding, schedules, and platform records).

1.2 Key Functional Requirements

- Patients can search doctors by specialization and view real-time slot availability.
- Patients can book, reschedule, or cancel an appointment slot.
- Doctors can define/update their available time slots and view their daily schedule.
- Doctors can issue a digital prescription linked to a completed consultation.
- Patients can view and download their prescription history.
- The system sends automated notifications for booking confirmations, reminders, and prescription availability.
- Patients pay a consultation fee online at the time of booking.

1.3 Key Non-Functional Requirements & Challenges

- Security & Privacy: Patient health data and prescriptions are sensitive (PHI) and require strict access control and encrypted storage/transmission.
- Performance & Scalability: Slot booking must handle concurrent requests during peak hours (e.g., morning booking rush) without double-booking a slot.
- Availability: Booking and prescription services must remain independently available — a failure in the payment gateway should not block a doctor from issuing a prescription.
- Usability: The booking flow must be simple enough for patients across a wide range of technical familiarity.

2. Architectural Style Analysis

Three candidate architectural styles were evaluated against the requirements identified above: Layered Architecture, Microservices Architecture, and Client-Server Architecture.

Style	Pros (relative to this system)	Cons (relative to this system)
Layered Architecture	Simple to build and understand; clear separation between UI, business logic, and data; suitable for a small initial release.	All modules deploy and scale together — a spike in slot-booking traffic forces scaling the entire application, including unrelated modules like prescriptions.
Microservices Architecture	Each capability (booking, prescriptions, payment, notifications) can scale and fail independently; supports strict data-access boundaries per service, which helps isolate sensitive health records; independent deployment allows faster iteration on individual features.	Higher operational complexity (service discovery, inter-service communication); introduces network latency between services; requires careful design for data consistency across services.
Client-Server Architecture	Centralized control and straightforward data consistency; simple to deploy for a small user base.	Single point of failure — if the central server goes down, booking, prescriptions, and payments all become unavailable simultaneously; harder to scale specific hot paths like slot booking independently.

3. Architecture Selection

Selected Architecture: Microservices Architecture

“We chose the Microservices Architecture for the Telemedicine Slot Booking & Prescription Portal.” The platform combines several distinct, independently evolving capabilities — appointment booking, prescription management, payments, notifications, and authentication — each with different scaling, security, and availability needs. A microservices approach lets each concern be built, deployed, secured, and scaled as its own unit behind a single API Gateway, which best matches both the functional and non-functional requirements of a healthcare booking platform.

Why Microservices Fits Best

A **microservices approach** lets each concern be **built, deployed, secured, and scaled as its own independent unit**, rather than forcing all functionality into a single, tightly coupled codebase. This means:

1. **Independent Evolution** – The booking logic can be updated or redesigned without touching the prescription or payment services, reducing regression risk in a domain where reliability is critical.
2. **Targeted Scaling** – Services under heavy load (e.g., Slot Booking during health drives) can be scaled out independently, keeping response times fast without over-provisioning the entire platform.
3. **Fault Isolation** – A failure in the Notification Service (e.g., an SMS gateway outage) does not prevent students from booking appointments or receiving prescriptions — a crucial guarantee in a healthcare context.
4. **Technology Diversity** – Each service can use the most suitable technology stack (e.g., a document database for prescriptions, a relational database for bookings, a message broker for asynchronous notifications).

The Role of the API Gateway

All these services sit **behind a single API Gateway**, which acts as the unified entry point for both the **Student Portal** and the **Registrar Dashboard**. The API Gateway handles request routing, authentication token validation, rate limiting, and load balancing, so that clients interact with a clean, consistent interface while the internal microservices remain loosely coupled and independently manageable.

Alignment with Requirements

This architecture **best matches both the functional and non-functional requirements** of a healthcare booking platform:

- **Functionally**, it cleanly separates the booking, prescription, payment, and notification workflows that Students and Academic Registrars rely on.
- **Non-functionally**, it satisfies the platform's demands for **high availability** (fault isolation), **security** (independent compliance boundaries per service), **performance** (auto-scaling of hot services), and **maintainability** (small, focused codebases owned by individual teams).

For these reasons, Microservices Architecture is the most appropriate and future-proof choice for the Telemedicine Slot Booking & Prescription Portal.

4. Component Identification

Based on the selected microservices architecture, six backend components were identified (the first two — Order/Appointment handling and Payment — were the anchor components; the remaining four were derived from the functional requirements):

#	Component	Responsibility
1	API Gateway	Single entry point for client apps; routes requests to the appropriate backend service and enforces coarse-grained access control.
2	Authentication & User Profile Service	Registers/authenticates patients and doctors; manages profile data and role-based access.
3	Appointment / Slot Booking Service	Core booking logic: doctor availability, slot locking, booking, rescheduling and cancellation.
4	Prescription Management Service	Allows doctors to issue prescriptions and patients to view/download prescription history.
5	Payment Service	Processes consultation-fee payments and returns confirmation to the Booking Service.
6	Notification Service	Sends booking confirmations, reminders, and prescription-ready alerts via SMS/email/push.

5. UML Component Diagram

The diagram below shows the six backend components, the two client-facing actors, and five labelled interfaces using standard UML ball-and-socket notation (provided interfaces as balls, required interfaces as sockets) plus «use» dependency arrows for the notification interactions.

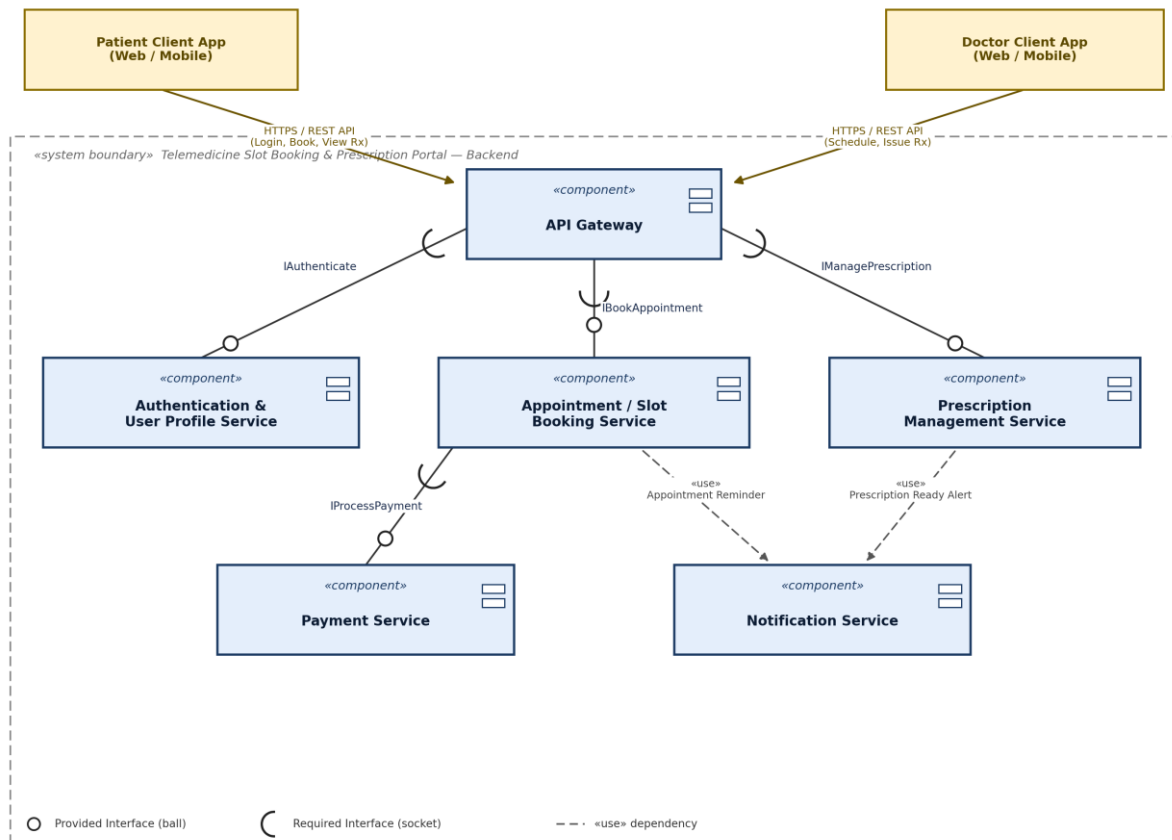


Fig. 1 — Component diagram of the Telemedicine Slot Booking & Prescription Portal backend

5.1 Interface Summary

Interface	Provider → Requirer	Purpose
IAuthenticate	Authentication Service → API Gateway	Validates patient/doctor credentials and issues session/access tokens.
IBookAppointment	Appointment Booking Service → API Gateway	Exposes slot search, booking, reschedule and cancellation operations.
IManagePrescription	Prescription Service → API Gateway	Exposes prescription creation (doctor) and retrieval (patient) operations.
IProcessPayment	Payment Service → Appointment Booking Service	Processes the consultation fee before a booking is confirmed.
INotify	Notification Service → Appointment Booking Service / Prescription Service	Used to trigger booking reminders and prescription-ready alerts («use» dependency).

6. Written Justification

“We chose the Microservices Architecture for the Telemedicine Slot Booking & Prescription Portal.”

Architectural Choice:

The system is built using a Microservices Architecture, composed of independently deployable services (Authentication, Appointment Booking, Prescription Management, Payment, and Notification) fronted by a single API Gateway.

Two Scenario-Related Reasons:

- Independent scaling for uneven load: Slot booking traffic spikes sharply during peak hours (e.g., early morning), while prescription retrieval and payment traffic stays comparatively flat. Microservices let the Appointment Booking Service be scaled independently — without over-provisioning the Prescription or Notification services — keeping infrastructure cost proportional to actual demand.
- Fault isolation for critical care workflows: A telemedicine platform cannot allow a non-critical failure to block a critical one. With microservices, if the Payment Service or Notification Service experiences downtime, doctors can still issue prescriptions and patients can still view existing bookings, because these capabilities run as separate services rather than one monolithic process.

Security Advantage:

Microservices allow strict, service-level access boundaries around sensitive data. The Prescription Management Service can be isolated with its own datastore and stricter authorization rules (only the treating doctor and the patient can access a given prescription), separate from booking and payment data. This limits the blast radius of a potential breach — compromising the Payment Service, for example, does not automatically expose patient health records, which is critical for regulatory compliance (e.g., HIPAA-equivalent healthcare data protection).

Performance Benefit:

Because each service is deployed and scaled independently behind the API Gateway, the Appointment Booking Service can be horizontally scaled with additional instances during peak booking windows without needing to scale the entire application. This keeps slot-booking latency low precisely when it matters most, while services with lighter, steadier load (such as Notification) run on minimal resources — improving overall responsiveness and infrastructure efficiency compared to scaling a single monolithic deployment.